Installation instructions

Installation Instructions

[Join Our Slack!]

1. Getting set up

Create a new folder for our project.
Open the folder in your IDE of choice (VSCode, Spyder, etc).
Download these images and add them to your folder. otherCar.png car.png
Create a Python file & save it with a file name.

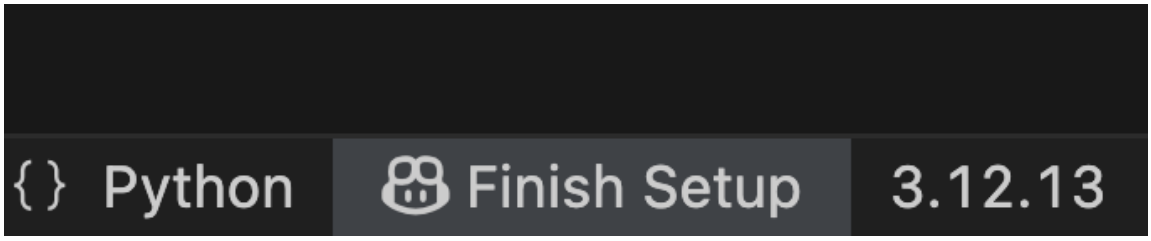
Check your version of Python:

Check which version of Python you have installed.
We're using Pygame, which only works with Python 3.8-3.12.
Create a terminal and type:

```
python --version
```

Skip to "**Installing Pygame"** if its within 3.8-3.12.
If your Python version is outside of 3.8-3.12,

1. Download Python 3.12 (scroll to bottom)
   https://www.python.org/downloads/release/python-3120/
2. Open the installer you just downloaded, and allow Python to be installed with all the default settings.
3. Look to the bottom of your IDE window. If it doesn't say 3.12, click on the version number (eg. 3.14), and select 3.12 from the top menu. Now the bottom should say 3.12.



Installing Pygame:

To install Pygame, create a terminal and type

```
pip install pygame
```

TIP: To check if you already have pygame installed, you can type in the terminal

```
python -m pygame.examples.aliens
```

This will allow you to play Pygame's example game.

Import Pygame and initialize it at the top.

```
import pygame
from pygame.locals import *
```

2. Errors

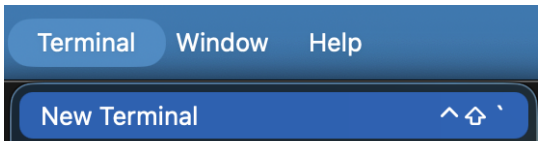
Follow the instructions on the right if you encounter any of the errors on the left column.

Ask for help if it doesn't work or at any time in this process!

Error	Try

zsh: command not found	• pip3 install pygame
	• If it still doesn't work, try these to verify you have python downloaded
	  ○ python
	  ○ python3

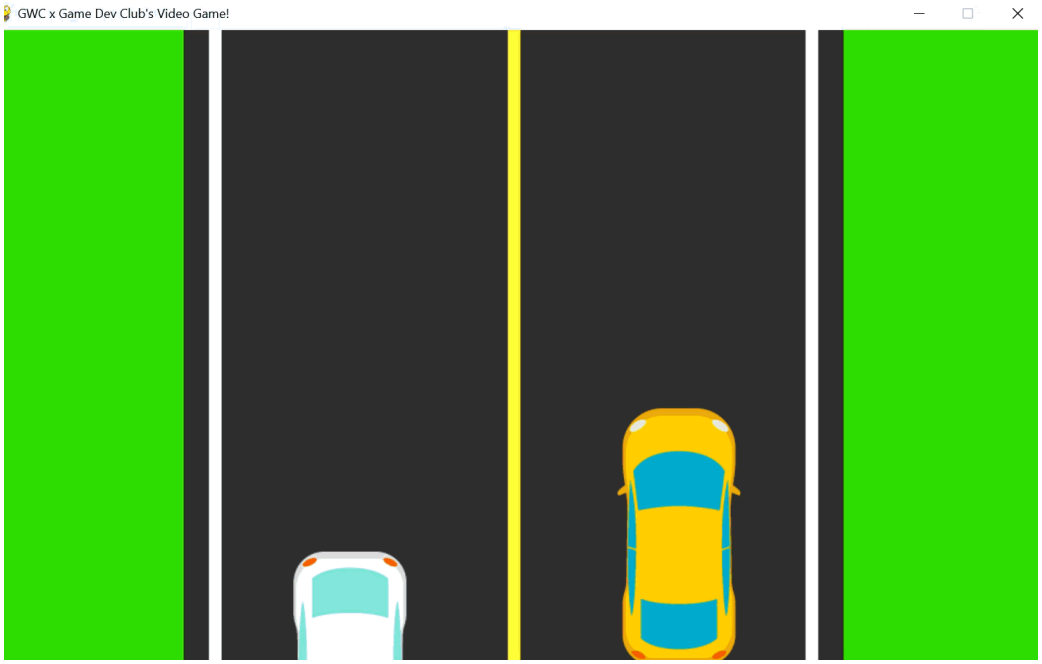
error: externally-managed-environment .. .. .. (etc)	Create a new virtual environment! 4. Go to your VSCode search bar. 1. Type "> Python: Select Interpreter" 2. Click "+ Create Virtual Environment…" 3. Click "Venv" 4. Choose any Python version you have (must be 3.8 - 3.12) a. 5. Close your terminal

	a. Top right of terminal panel 6. Reopen a new one  a. The first command in your terminal should end with "/.venv/bin/activate" 7. Reinstall pygame by entering this command: **a. python -m pip install -U pygame**
Error mentioning **SDL.h**	● You probably have a Mac! ● In your terminal: ○ **brew install sdl2 sdl2_image** ● Try **python -m pip install -U pygame** ○ Or: python -m pip install -U pygame
Sorry, extended image module required	● You probably have a Mac! Pygame has been downloaded already but there are some configuration issues. Try the command below & then retry the aliens test game, but if it doesn't work, don't sweat it. Our game will still work just fine! ● In your terminal: - **brew install sdl2 sdl2_mixer sdl2_gfx sdl2_image sdl2_net sdl2_ttf** → **use python –version to see which python version you have. Is it 3.8-3.12?**

Tutorial

GAME DEV WORKSHOP

GWC x Game Dev Club



Your game will run better than this low quality gif we promise...

tutorial

1. Getting set up

Create a new folder for our project.
Open the folder in your IDE of choice (VSCode, Spyder, etc).
Download these images and add them to your folder. otherCar.png car.png

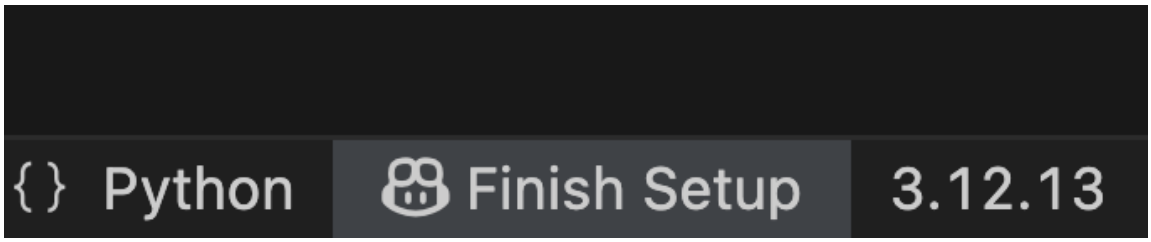
Check your version of Python (3.8-3.12):

We're using Pygame, which only works with Python 3.8-3.12.
Create a terminal and type:

```
python --version
```

Skip to "**Installing Pygame"** if it's within 3.8-3.12.
If your Python version is outside of 3.8-3.12,

1. Download Python 3.12 (scroll to bottom)
   https://www.python.org/downloads/release/python-3120/
2. Open the installer you just downloaded, and allow Python to be installed with all the default settings.
3. Look to the bottom of your IDE window. If it doesn't say 3.12, click on the version number (eg. 3.14), and select 3.12 from the top menu. Now the bottom should say 3.12.



Installing Pygame:

Create a Python file.
Install Pygame by creating a terminal and typing:

```
pip install pygame
```

TIP: To check if you already have pygame installed, you can type in the terminal

```
python -m pygame.examples.aliens
```

This will allow you to play Pygame's example game.

Next, create a new folder for our project and place a Python file inside it. Import Pygame and initialize it at the top.

```
import pygame
from pygame.locals import *

pygame.init() #initializes pygame

size = width, height = (1000, 600)
screen = pygame.display.set_mode(size)#creates our game window

pygame.quit() #closes the application when we're done playing
```

At this point, the game ends immediately after running the application. We need to put some logic in between the commands.

```
import pygame
from pygame.locals import *
```

```
pygame.init()

size = width, height = (1000, 600)
screen = pygame.display.set_mode(size)

#waits for user to click the exit button to stop playing
running = True
while(running):
    for event in pygame.event.get():
        if (event.type == QUIT):
            running = False

pygame.quit()
```

Here we have created a **game loop**. This loop is one big while loop that controls the core logic of our game. Every "frame" is one iteration of this loop.
In order to quit the game, we need to know when the user has clicked the "x" icon. This is done with **events**. Events are certain actions that are fired at some point during our game (such as user inputs, window resizing, etc.). In this case, we listen to the QUIT event, and break out of the loop.

TIP: If you want to change the name of the app window, use
`pygame.display.set_caption()`

```
pygame.display.set_caption("GWC x Game Dev Club's Video Game!")
```

2. Drawing the Road

Note:

> As you follow along, run **python <your-file-name>.py** in your terminal to see the graphics you just created! Click the X button on the popup screen to close the game.
>
> As you update your code, you'll have to run **python <your-file-name>.py** to see the most recent version.

Now, let's add some more interesting stuff to our game.
First, let's add a grass background before the game loop.

```
#uses RGB value to give us a grassy green background
```

```
screen.fill((60, 220, 0))

#actually updates our game
pygame.display.update()

while(running):
    . . .
```

We'll be adding colors using RGB values. The final color is added based on the value for the red, green, and blue color channels, with each value being from 0-255. Channels with higher values have stronger colors.

Without `pygame.display.update()`, the display of our game won't update and our background won't be shown. We will need to call this function again later when we update the display again.

Second, let's draw our road. We need to first create a variable for the width of the road.

```
road_width = int(width / 1.6)
```

Instead of just setting `road_width` to equal a specific number, we use our `width` variable, the width of the screen . That way, if we decide to change the size of our screen later, we will not have to also manually change the width of the road. It will change dynamically.

Now we can actually draw the road.

```
pygame.draw.rect(
    screen,
    (50, 50, 50), #color of road
    ((width/2 - road_width/2), 0, road_width, height) #x, y, width, height
)
```

The x coordinate is defined as `(width/2 - road_width/2)` as that centers the road in the middle of the screen. Pygame positions objects based on the x & y coordinates of the object's top left corner, which is why an x coordinate of width/2 would not center the road.

Third, let's draw the yellow road marker in the middle of the road. This is done very similarly to drawing the road.

```
roadmark_width = int(width / 80)
```

```
pygame.draw.rect(
```

```
    screen,
    (255, 240, 60),
    (width/2 - roadmark_width/2, 0, roadmark_width, height)
)
```

Fourth, we will add the two white lines at the edges of the road. We do not need to create a separate variable for the width, but they will be drawn similarly to the last two steps.

```
#leftmost line
pygame.draw.rect(
    screen,
    (255, 255, 255),
    ((width/2 - road_width/2) + roadmark_width*2, 0, roadmark_width,
height)
)

#rightmost line
pygame.draw.rect(
    screen,
    (255, 255, 255),
    ((width/2 + road_width/2) - roadmark_width*3, 0, roadmark_width,
height)
)
```

`roadmark_width*2` or `roadmark_width*3` is added to the x coordinate to push the line to the edge of the road.

3. Adding Images

We will now place our car images into the game. Download the car images otherCar.png car.png and place them into your project folder if you haven't already.

Load the images of the cars. Right above the while loop / game loop, type.

```
#load player image
car = pygame.image.load("car.png")

#load enemy image
car_enemy = pygame.image.load("otherCar.png")
```

We need to place this code above our game loop since we only want these images to be loaded in once.

Next, place the cars in the correct lanes. Let's first create variables for the right and left lanes to make this more readable.

```
right_lane = width/2 + road_width/4
left_lane = width/2 - road_width/4
```

Now, we can place the cars in their correct lanes.

```
# fetch the location of the images
car_location = car.get_rect()
car_enemy_location = car_enemy.get_rect()

#places the center of the cars in the correct lane
# player car will be on the bottom of the right lane
car_location.center = right_lane, height * 0.8 #x, y coords

# enemy car will be on the top of the left lane
car_enemy_location.center = left_lane, height*0.2
```

`car.get_rect()` and `car_enemy.get_rect()` store the coordinates of each car as a tuple.

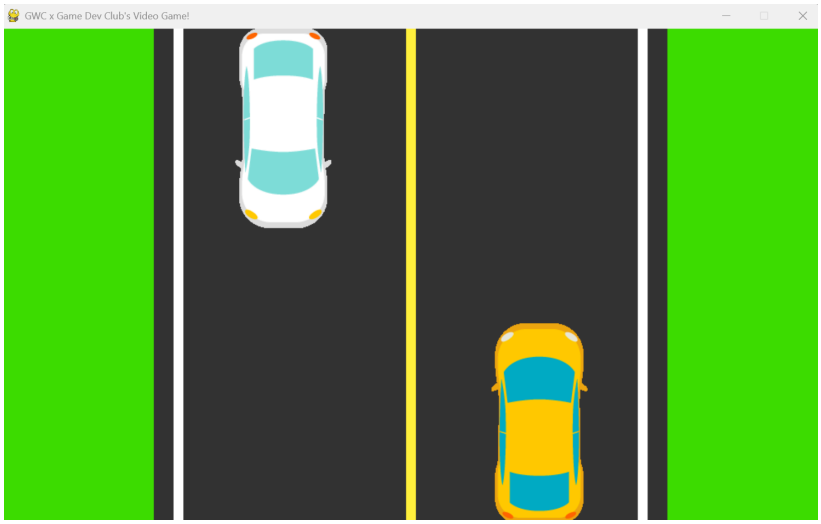
We have loaded and placed the images, but we now need to draw them. At the **end of our game loop**, type

```
    screen.blit(car, car_location)
    screen.blit(car_enemy, car_enemy_location)

    #update our game again now that we have a new drawing
    pygame.display.update()
```

`screen.blit(image, location)` is a function that draws one surface onto another at specific coordinates. It's basically a drawing function.

Checkpoint: If your game looks like this so far, you're on the right track!



4. Game Logic

We will now add the game controls inside of our while loop.

Let's add controls so that our car can move to the left lane.

```
#game loop we made previously
while(running):
    for event in pygame.event.get():
        #quit event
        if (event.type == QUIT):
            running = False

        #Adds the controls ↓
        #selects ALL keys on our keyboard
        if (event.type == KEYDOWN):
            #if the key pressed is either the a key or left arrow key...
            if(event.key in [K_a, K_LEFT]):
                #moves the player to the left lane
                car_location = car_location.move([-int(road_width/2), 0])
#x, y coords
```

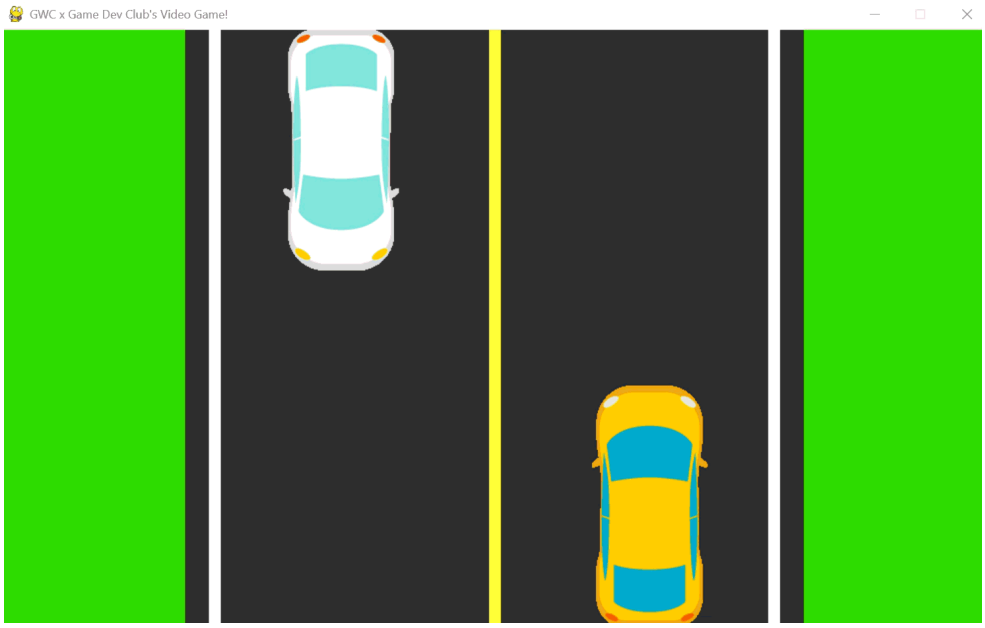
`K_a` and `K_LEFT` stand for the a key and left arrow key. Also notice that we are subtracting the x coordinate.

We can now do the same to move our player car to the right lane. We will use the d or right arrow key to move.

```
if(event.key in [K_d, K_RIGHT]):
      car_location = car_location.move([int(road_width/2), 0])
```

Make sure to **add** the x coordinate this time instead of subtracting it.

While the game controls do work now, something strange is happening when we move...



Instead of the player moving, the image is actually getting duplicated each time. This is because the previous frame is never actually cleared, and stays the same.

To fix this, we need to place **all** of the graphics, including the background and the road, at the end of the game loop right before our call to `screen.blit()`. This way, we redraw *everything* every frame instead of keeping the data from the previous frame.

```
# game loop
while(running):

    ALL OF OUR GAME LOGIC
    . . .

    ALL OF OUR EVENTS
    for event in pygame.event.get():
```

```
        . . .


    PLACE THE GRAPHICS HERE!!
    screen.fill((60, 220, 0)) # Also clears the previous frame data
    pygame.draw.rect(
        screen,
        (50, 50, 50),
        (width/2 - road_width/2, 0, road_width, height)
    )

    REST OF GRAPHICS…
    . . .

    screen.blit(car, car_location)
    screen.blit(car_enemy, car_enemy_location)
    pygame.display.update()

# end of game loop
```

Next, let's animate the enemy. We should first move the enemy's location down by one pixel on every iteration of our game loop. Place this code at the top of our game loop.

```
#game loop
while(running):

    # move the enemy
    car_enemy_location[1] += 1
```

Remember that `car_enemy_location` gives the enemy's coordinates as a tuple. Since we only want to change the y coordinate of the car, we need to access the second value of the tuple. This is why we type `car_enemy_location[1]`.

Right now, once the enemy drives off the screen, it is gone forever 😢. To fix this, add a conditional that says once the enemy's y coordinate is greater than the screen's height, then place it a little above the screen.

```
# relocates the enemy when it drives off the screen
    if(car_enemy_location[1] > height):
        car_enemy_location[1] = -200
```

The enemy is currently only in the left lane. To make our game actually challenging, we need it to move between lanes. To do this, we will make it so the enemy randomly switches between lanes.
First, import the random module.

```
import random
```

Next, update the enemy's logic.

```
#game loop
while(running):
    car_enemy_location[1] += 1

    if(car_enemy_location[1] > height):
        #if 0 is chosen as the random number...
        if (random.randint(0, 1) == 0):
            #the enemy moves to the right lane
            car_enemy_location.center = right_lane, -200

        #if 1 is chosen as the random number...
        else:
            #the enemy moves to the left lane
            car_enemy_location.center = left_lane, -200
```

Instead of the enemy's y coordinate just moving to `-200` when it goes off screen, it will first choose whether to go to the right lane or left lane, then move.

8. Collisions

Currently, if the enemy and the player collide, nothing happens. We need to add the end game condition.

Below the code for the enemy's location but above the event listeners, we need to add the collision logic.

```
#if the cars are in the same lane AND the enemy car hits the front of the
player...
    if (car_location[0] == car_enemy_location[0])
      and (car_enemy_location[1] > car_location[1] - 250):
        print("GAME OVER!")
        running = False
```

The first part of our conditional is asking if the player car and the enemy car have the same x coordinates. This means that they are in the same lane. The second part is asking if the enemy's y coordinate is greater than the very front of the player car. This means that the enemy has hit the front of the player's car.
When we set running to false, this breaks out of our while loop and brings us to the very last line of code in our program which is `pygame.quit()`. This will force the app window to close.

🎉 CONGRATULATIONS! YOUR GAME IS NOW COMPLETE!!!! 🎉

6. Mods

While the base game is complete, the game itself is very basic. Try implementing some mods to the game to make it more interesting.
Here are some ideas:

1. Create your own assets for the cars
   Hint: the current images are 250 x 250 pixels.

2. Make it so the player car can not drive off the side of the road

3. Increase the enemy's speed as game progresses

4. Add a score
   Hint: There are multiple ways to implement this mod and some are much tricker than others. Think about how we have implemented other mechanics to our game, and try to do it in a similar way.

5. Add text to the screen that displays the enemy's current speed or the current score
   Hint: This mod will require some Googling and implementing Pygame features we did not go over during the workshop.